# CoralNPU on AWS F2: Gemma 3 1B and FunctionGemma 270M

Inference status report

Ergodex – coralnpu_bonsai, branch `model-kernels-firmware`

2026-10-14

## 1 Summary

Both models now run end to end on Alex's F2 image (`agfi-075a98d106eb1b6bf`, small shell `0x10212415`) using hand-written CoralNPU kernels under `models/`. On the FPGA, FunctionGemma 270M generates *token-identical* output to the MLX bf16 reference for the chat prompt used throughout this report. Throughput on the image is bound by external memory: the core streams about 0.7 bytes per cycle at 25 MHz, so a FunctionGemma decode step takes about 28 s and a Gemma 3 1B step is expected to take 50–60 s. Table 2 lists the measured time to first token (TTFT) and tokens per second for the prompt lengths run so far; runs still in progress are marked and the report is regenerated from the result files (`report/collect.py`).

## 2 Setup

**FPGA** EC2 `f2.6xlarge` `fpga-dev-f2` (`i-00a83cab3c81ac10f`), slot 0, Alex's AFI. Core: `CoralF2Axi` at 25 MHz, 8 KiB ITCM, 32 KiB DTCM, RVV (VLEN 128, Zve32f) and the Zvt matrix engine with int8 and f32 modes only (no bf16 anywhere). DDR (1 GiB window at `0x80000000`) holds weights, HBM (`0xC0000000`) holds activations, the KV cache and RoPE tables.

**Software** One ELF per operator (`models/kernels`), built on the instance with clang-18 and lld (`models/firmware/build.sh`); the host runtime (`models/host/runtime.py`) sequences the kernels over PCIe through the AWS FPGA SDK.

**Weights** Gemma 3 1B: Q4_0 from `gaunernst/gemma-3-1b-it-int4-gguf`, kept exactly (int4 codes in groups of 32 with f16 scales). FunctionGemma 270M: bf16 safetensors from the public MLX mirror `mlx-community/functiongemma-270m-it-bf16`, quantized to the same Q4_0 blocks. Activations are int8 per 32-wide group; residual stream, norms, attention and the KV cache are f32.

**MLX reference** `mlx-lm` on this Mac (Apple M2 Max, 32 GiB), greedy decoding, same chat template and tokenizer. `mlx-community/functiongemma-270m-it-bf16` and `mlx-community/gemma-3-1b-it-qat-4bit`.

**Other paths** The IREE / coralnpu-compiler path (branch `fpga-workflow`: F2 PCIe backend, int4 Gemma export, MPACT golden) is implemented and verified on tiny models on the Mac but has not yet been built on the instance, so no MPACT or compiler-path numbers are reported here. The NumPy reference (`models/tests/reference.py`) reproduces the device arithmetic and is the per-stage golden for the kernels.

# 3 Correctness

Every kernel matches its NumPy reference on the hardware (`models/tests/fpga_kernels_test.py`); the matrix-engine matvec agrees to $1.2 \times 10^{-7}$. A full FunctionGemma decode step traced stage by stage against the reference (`models/tests/trace_compare.py`) agrees to about $10^{-3}$ relative at layer 0, drifts to a few percent in the middle layers, and ends with the same top-4 logits in the same order. Table 1 compares the generated text for the chat prompt *"What is the capital of France?"* (16 prompt tokens including `<bos>` and the chat template).

| Model | Backend | Generated text | Tokens match MLX |
|---|---|---|---|
| FunctionGemma 270M | MLX functiongemma-270m-it-bf16 | The capital of France is Paris. | reference |
| FunctionGemma 270M | FPGA (F2, native kernels) | The capital of France is Paris. | yes |
| Gemma 3 1B | MLX gemma-3-1b-it-qat-4bit | The capital of France is Paris. | reference |
| Gemma 3 1B | FPGA (F2, native kernels) | *run in progress* | – |

Table 1: Chat prompt output, FPGA native kernels versus MLX.

# 4 Performance

## 4.1 FPGA (25 MHz image)

TTFT is the wall time of the prefill pass (the prompt is processed in chunks of up to 16 tokens, each chunk costing about one decode step because the weights are streamed once per chunk). Decode tokens per second is measured over the generated tokens after the first. Synthetic prompts ("isl$N$") are exactly $N$ tokens and generation ignores the end-of-sequence token so the output length (OSL) is fixed.

| Model | Prompt | ISL | OSL | TTFT (s) | s/token | tok/s |
|---|---|---|---|---|---|---|
| FunctionGemma 270M | isl8 | 8 | 16 | 48.7 | 28.0 | 0.0358 |
| FunctionGemma 270M | chat | 16 | 8 | 66.8 | 27.7 | 0.0361 |
| FunctionGemma 270M | isl32 | 32 | 16 | 137.6 | 28.1 | 0.0356 |
| FunctionGemma 270M | isl64 | 64 | 16 | 289.7 | 29.3 | 0.0341 |
| FunctionGemma 270M | isl96 | 96 | 16 | 482.9 | 29.2 | 0.0343 |
| Gemma 3 1B | *runs in progress* | | | | | |

Table 2: FPGA results. Entries marked "in progress" are regenerated when the runs finish.

**Where the time goes.** The bandwidth microbenchmark (`models/tests/membench.py`) measures 0.65–0.77 B/cycle for reads and writes to both DDR and HBM, independent of vector width: about 24 core cycles per 16-byte load, i.e. 16–19 MB/s at 25 MHz. The 1B model streams about 350 MB of int4 weights per token plus 150 MB for the tied embedding at the logits, giving a floor of roughly 30 s per token before any compute; the 270M model streams about 65 MB per token plus the same 150 MB logits matrix, so its decode is dominated by the vocabulary projection. The matvec kernel runs at about 645 cycles per 512 multiply-accumulates in decode (one 16-byte activation load per 16 K values, two weight loads and a scale load per 32-wide group), close to that floor. In the first FunctionGemma run, 637 matvec launches accounted for 120 s of the 235 s total; the scalar GELU kernel for 12 s.

### 4.2 MLX on Apple Silicon

| Model | MLX checkpoint | ISL | OSL | TTFT (s) | tok/s |
|---|---|---|---|---|---|
| FunctionGemma 270M | functiongemma-270m-it-bf16 | 16 | 8 | 1.130 | 53.0 |
| Gemma 3 1B | gemma-3-1b-it-qat-4bit | 16 | 8 | 0.960 | 56.9 |

Table 3: MLX reference runs on the Mac for the same prompt.

### 4.3 Projection to 1 GHz silicon

The kernels issue roughly 11 instructions per matrix step (64 int8 MACs in decode). If a silicon implementation removed the external-memory bottleneck, instruction issue would bound decode at about 8 tokens/s for Gemma 3 1B and about 60 tokens/s for FunctionGemma 270M at 1 GHz; with the measured F2 bus behaviour scaled by clock alone the numbers would be 40× the FPGA figures. The MPACT simulator path (`tools/estimate_tps.py` on branch `fpga-workflow`) is the intended tool for a tighter estimate and has not been run yet.

## 5 Hardware findings during bring-up

- **Memory window base.** The core reads wrong data from the first bytes of the HBM window (`0xC0000000`) although host access to the same bytes is correct; the first 64 KiB of each window is now left unused. This was the only functional bug found in the integration: with it fixed, the device matched the reference at every stage.
- **Activation layout matters more than compute.** Gathering sixteen activation rows with strided loads cost 16 bus transactions per matrix step; a k-major layout written by the quantizer (one contiguous load per row) cut the matvec from 50,720 to 7,669 cycles on the test shape, and the decode fast path (one load per 16 K values, rows derived with register slides) brought decode to 645 cycles per 512 MACs.
- **Scalar loops are expensive.** Each scalar access to external memory costs the same 24 cycles; the vocabulary top-$k$ was rewritten as vector chunk maxima plus an exact scan (262,144 logits in 1.8M cycles, 73 ms).
- **Matrix engine.** The int8 `vtmms` path with four K values per instruction works as documented in the repository's VME tests; the signed-by-unsigned mode consumes raw Q4_0 nibbles directly.

## 6 Next steps

1. Finish the ISL/OSL sweeps for both models (in progress on the instance) and refresh this report and pull request #10.
2. Build the coralnpu-compiler path on the instance and compare its F2 backend against the native kernels on the same prompts; run MPACT for the 1 GHz estimate.
3. Reduce weight traffic further: f16 group scales (halves scale reads), fusing the gated GELU into the matvec drain, and batching 16 decode streams to use all tile rows.